# Fail-Safe In-Orbit Reprogramming of a Spacecraft Controller

Hazard Analysis

| | |
|---|---|
| Reference | TBD **[TBD-1]** |
| Issue / revision | Issue 1, Revision 0 — DRAFT |
| Date | 10 October 2026 |
| Author | Andrew Toma |
| Open items | 2 TBD, 3 TBR (App. A) |

**Document change record**

| Issue | Rev. | Date | Change |
|---|---|---|---|
| 1 | 0 | 2026-10-10 | First draft |

## 1.0 Purpose and Scope

This document is the preliminary hazard analysis of the System: its hazards, their causes and severity, and the controls proposed against them.

The hazard analysis is delivered once per safety phase, each closed by a safety review [2, Secs. 3.1 and 4]:

**Phase 0** Concept complete: the top hazards and their causes.
**Phase 1** Requirements nearly complete: all known hazards, with their causes, controls and risk.
**Phase 2** Design mostly complete: the approved controls, and how each is verified.
**Phase 3** Testing complete: proof that the verifications passed.

This document is the first delivery: phases 0 and 1, which small projects combine [2, Sec. 4]. The controls are proposals that the requirements turn into rules; each already names its verification method [7, App. F.3, p. 348].

The document also decides which software is safety-critical, which the Software Classification left open [6, Sec. 4.0]. It covers the software items of that classification [6, Sec. 3.0] in operation: all controller modes, during and between contacts. Bench and build tools are covered where they can reach operation.

## 2.0 Documents

[1] NASA, *Software Assurance and Software Safety Standard*, NASA-STD-8739.8B, 2022. Causes cite App. A, Table 2, pp. 62–68.
[2] NASA, *NASA Software Engineering Handbook*, NASA-HDBK-2203, Version D, Topic 8.58 “Software Safety and Hazard Analysis”.
[3] NASA, *NASA Software Engineering Handbook*, NASA-HDBK-2203, Version D, Topic 8.05 “SW Failure Modes and Effects Analysis”.
[4] NASA, *Fault Tree Handbook with Aerospace Applications*, Version 1.1, August 2002.
[5] *Fail-Safe In-Orbit Reprogramming of a Spacecraft Controller — Concept of Operations*, Issue 1, Revision 0, Draft.
[6] *Fail-Safe In-Orbit Reprogramming of a Spacecraft Controller — Software Classification*, Issue 1, Revision 0, Draft.
[7] NASA, *NASA Software Safety Guidebook*, NASA-GB-8719.13, 2004.
[8] NASA, *NASA Systems Engineering Handbook*, NASA/SP-2016-6105 Rev 2, 2016.

[9] NASA, *Fault Management Handbook*, NASA-HDBK-1002, Draft 2, 2 April 2012. A draft, not a released standard.

# 3.0 Ground Rules

These rules set the boundary, the failures included, the fault tolerance required and how failures are modelled and named, before the analysis starts [3, Sec. 3.1; 4, pp. 23–24].

## 3.1 Definitions and Boundary

- **Hazard:** a state with the potential to cause harm [1, p. 13]. Here, harm is the loss of the controller's operability, observability or correctability.
- **Boundary:** the radio equipment and the spacecraft power supply lie outside the System [5, Sec. 1.2.2]. Their failures enter as causes and are not analysed further.
- **Physical tampering:** excluded; nobody can reach the spacecraft (Assumption A-1).

## 3.2 Failures Included

The failures below are taken from the software causes of hazards [1, App. A, Table 2, pp. 62–68].

| Domain | Failures |
|---|---|
| Software | Hang, crash, buffer overflow or invalid state in any software item |
| Data | Message corrupted, lost, repeated or late on any link; full buffer |
| Memory | Write cut short by power loss; single-event upset (a bit flipped by radiation, A-5); corrupted value that survives a restart |
| Timing | Watchdog timeout; restart loop; onboard clock wrong, drifting or reset |
| Power | Unannounced loss at any moment (A-2) |
| Hardware link | Physical failure of a data-link connection or the reprogramming link |
| Operator | Invalid, out-of-sequence or mistimed telecommand (A-4) |
| Security | Means of proving origin disclosed |

## 3.3 Analysis Rules

- **Controls:** "means of reducing the risk of exposure to a hazard" [1, p. 13]: a check, a backup, an automatic response or an operator procedure.
- **Counting failures:** one cause that disables several controls at once counts as one failure.
- **Severity:** how bad the outcome is if the hazard happens. Controls do not change it: they make a hazard less likely, not less bad.
- **Fault tolerance:** "the ability to perform a function in the presence of any of a specified number of coincident, independent failure causes of specified types" [9, p. 18]. The types are the failures included above. The number depends on severity: a catastrophic hazard must need at least three failures together, a critical hazard at least two [3, Sec. 3.1] **[TBR-1]**. Each hazard is checked by finding the smallest set of failures that causes it.
- **Likelihood:** not rated; it depends on the design **[TBD-2]**.
- **Safety effort:** full [7, Sec. 3.2.1, p. 44], chosen by the author on 10 October 2026. A moderate effort suits hazards that operators have time to answer [7, Sec. 3.2.2, p. 44]; between contacts, no operator can (A-3). At this stage, the full effort makes a fault tree mandatory and a failure modes analysis highly recommended [7, Table 6-3, p. 107].
- **Assumptions:** those of the ConOps, A-1 to A-5; this analysis adds none.

**Legend: severity**, as defined in [7, Table 2-2, p. 27], then for the System

**CATASTROPHIC** "Loss of human life or permanent disability; loss of entire system; loss of ground facility; severe environmental damage". The controller is lost for the rest of the mission: it cannot run, or can no longer be commanded, observed or corrected.

**CRITICAL** "Severe injury or temporary disability; major system or environmental damage". The controller acts against the operator's intent, or cannot be commanded, observed or corrected until the operator acts at a later contact.

**MODERATE** "Minor injury; minor system damage". Part of a function is lost, such as some health reports or one update attempt; the controller still runs and obeys the operator.

**NEGLIGIBLE** "No injury or minor injury; some system stress, but no system damage". Nothing is lost; the operator has extra work or gets data a contact later.

### 3.4 Naming

- **Hazards:** H-1 to H-17, in order of severity.
- **Fault tree:** gates G01 to G06; primary events B01 to B10. No identifier is ever reused.
- **ConOps numbers:** objectives (O), assumptions (A), constraints (C), risks (R), capabilities and scenarios refer to the ConOps [5, Secs. 1.1.2, 3.1, 3.5, 6.0 and 8.0]. A control with a scenario number comes from the ConOps; one without is new.
- **Software role** [2, Sec. 3.2.2]: CAUSE, a software fault can lead to the hazard; CONTROL, software prevents it; MITIGATION, software limits its effect; MONITOR, software detects and reports it.
- **Verification method** [8, Sec. 5.3.1.2.1, p. 92]: TEST, using the product to obtain detailed data; DEMONSTRATION, using it to show the control works, without detailed data; INSPECTION, examining the product or its records; ANALYSIS, modelling or calculation.

## 4.0 Preliminary Hazard List

The hazards in order of severity [**TBR-2**]:

| ID | Hazard | Severity |
|---|---|---|
| H-1 | The controller is left with no firmware it can run, and cannot be restored | CATASTROPHIC |
| H-2 | Firmware from someone other than the operator runs on the controller | CATASTROPHIC |
| H-3 | New firmware can no longer reach the controller | CATASTROPHIC |
| H-4 | The controller can no longer be commanded or observed | CATASTROPHIC |
| H-5 | Someone other than the operator commands the System | CATASTROPHIC |
| H-6 | A command is acted on in a form or at a time the operator did not intend | CRITICAL |
| H-7 | Wrong firmware runs on the controller: damaged, out of date, or with a hidden defect | CRITICAL |
| H-8 | The controller stops working between contacts and is not brought back | CRITICAL |
| H-9 | The ground is shown a false state of the System | CRITICAL |
| H-10 | A fault between contacts never reaches the ground | CRITICAL |
| H-11 | A valid command is not acted on, and the ground is not told | CRITICAL |
| H-12 | A test function acts outside a test | CRITICAL |
| H-13 | A fault response is taken when there is no fault | MODERATE |

| ID | Hazard | Severity |
|---|---|---|
| H-14 | An update cannot be completed | MODERATE |
| H-15 | Health reports are lost | MODERATE |
| H-16 | Reports reach the ground later than planned | NEGLIGIBLE |
| H-17 | A command is refused with a wrong or unclear reason | NEGLIGIBLE |

## 5.0 Top-Down Analysis

Figure 1 develops the worst hazard, H-1. Each box states what fails and when [4, p. 50].

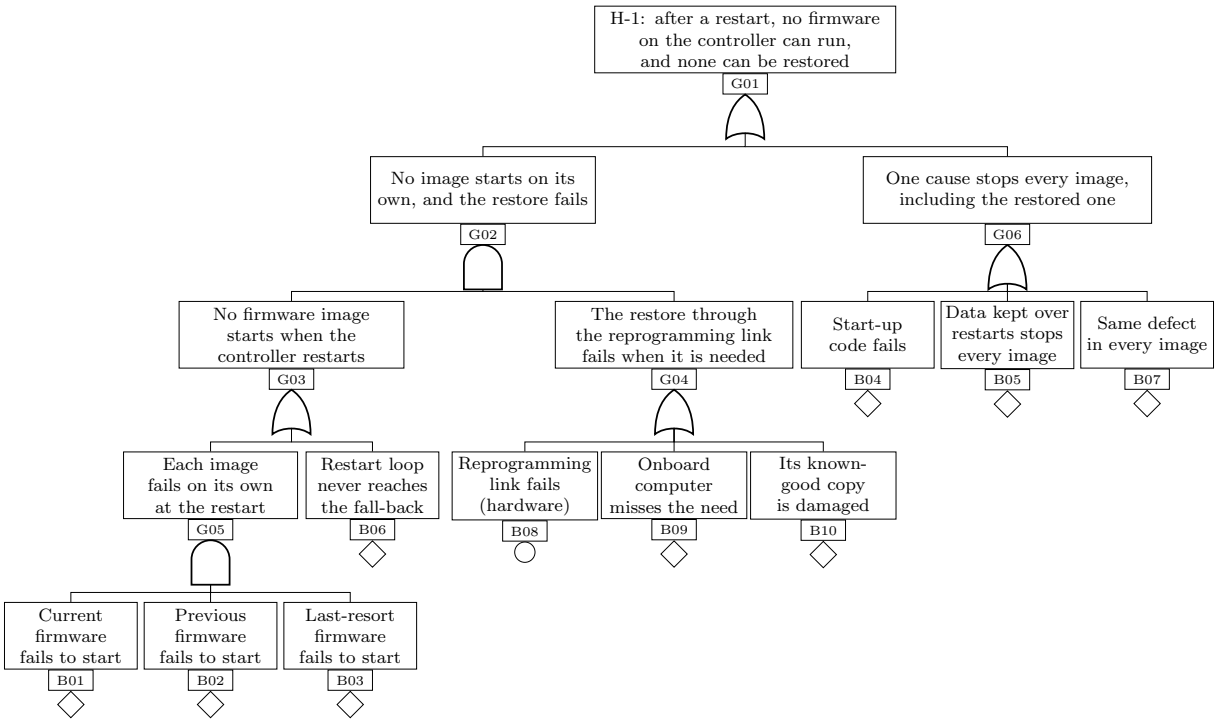



Figure 1: Fault tree of H-1. Rectangle: event developed by the gate below it. Circle: basic event, needing no further development. Diamond: undeveloped event; the design that would develop it does not exist yet. Flat base: AND gate. Curved base: OR gate. ID box: gate (G) or primary event (B). Symbols as in [4, Table 4-1, p. 34] and [4, Fig. 4-1, p. 33].

A minimal cut set is the smallest combination of primary events that causes the top event [4, Sec. 1.3, p. 3]. H-1 has three: one event (B04, B05 or B07); two events (B06 with one of B08 to B10); four events (B01, B02 and B03 with one of B08 to B10). Until the design shows otherwise, B04, B05 and B07 also stop the image the restore puts back, so each causes H-1 alone.

The table below gives the same count for every catastrophic and critical hazard, with the proposed controls working.

| Hazard | Severity | Needed | Fewest | Meets rule |
|---|---|---|---|---|
| H-1 | CATASTROPHIC | 3 | 1 | NO |
| H-2 | CATASTROPHIC | 3 | 1 | NO |
| H-3 | CATASTROPHIC | 3 | 1 | NO |
| H-4 | CATASTROPHIC | 3 | 1 | NO |
| H-5 | CATASTROPHIC | 3 | 1 | NO |
| H-6 | CRITICAL | 2 | 1 | NO |
| H-7 | CRITICAL | 2 | 2 | YES |
| H-8 | CRITICAL | 2 | 1 | NO |
| H-9 | CRITICAL | 2 | 1 | NO |
| H-10 | CRITICAL | 2 | 2 | YES |
| H-11 | CRITICAL | 2 | 2 | YES |
| H-12 | CRITICAL | 2 | 2 | YES |

**Legend.** Needed: failures the fault tolerance rule requires before the hazard can occur (Section 3.3). Fewest: failures that cause it now. Meets rule: YES when Fewest is at least Needed.

The single failures that break the rule:

- H-1: one cause that stops every image, including the restored one.
- H-2 and H-5: the means of proving origin is disclosed.
- H-3: a defect in the onboard computer's update path, which cannot be corrected.
- H-4: one software defect that cuts both data-link connections.
- H-6: a wrong onboard clock, or a bit error in a held command after its check.
- H-8: the same defect in every image.
- H-9: a wrong onboard clock.

## 6.0 Bottom-Up Analysis

Before a design exists, the components examined are the System's functions [3, Sec. 4]. Each function is examined for each way it can fail, and each failure is traced to the hazard it leads to.

| Function | OMITTED | INCORRECT | UNWANTED | MISTIMED |
|---|---|---|---|---|
| *Telemetry (Capability-1)* | | | | |
| Make health and event reports | H-10, H-15 | H-9 | H-4, H-15 | H-9 |
| Time-tag and store reports | H-10, H-15 | H-9 | H-15 | H-9 |
| Send reports down | H-10, H-15 | H-9 | H-16 | H-16 |
| Keep received reports on the ground | H-10, H-15 | H-9 | NONE | NONE |
| *Command handling (Capability-2)* | | | | |
| Prepare and check commands on the ground | H-6 | H-6 | H-6 | H-6 |
| Check commands onboard | H-5, H-6 | H-6, H-11, H-17 | NONE | H-4, H-6, H-11 |
| Hold armed and scheduled commands | H-11 | H-6 | H-6 | H-6 |
| Run commands | H-11 | H-6 | H-5, H-6 | H-6 |
| Answer and log commands | H-11 | H-9, H-17 | H-9 | H-16 |
| *Fault handling (Capability-3)* | | | | |
| Detect faults | H-8, H-10 | H-10, H-13 | H-13 | H-8 |
| Respond to faults | H-8 | H-1, H-4, H-13 | H-13 | H-8 |
| *Update (Capability-4)* | | | | |
| Prepare and check firmware releases | H-7 | H-3, H-7 | H-7 | NONE |
| Receive and check firmware | H-1, H-2, H-7 | H-2, H-3, H-7, H-14 | H-14 | H-7 |

| Function | OMITTED | INCORRECT | UNWANTED | MISTIMED |
|---|---|---|---|---|
| Install firmware and switch | H-14 | H-1 | H-6 | H-1 |
| Judge and confirm new firmware | H-13 | H-3, H-7 | H-7 | H-7, H-13 |
| Fall back to working firmware | H-1, H-8 | H-1, H-7 | H-13 | H-8, H-13 |
| Restore through the reprogramming link | H-1 | H-1, H-2, H-7 | H-13 | H-1 |
| *Test (Capability-5)* | | | | |
| Inject faults | NONE | NONE | H-12 | H-12 |

**Legend: failure mode.** OMITTED: not performed. INCORRECT: performed with a wrong result. UNWANTED: performed when it must not be. MISTIMED: performed too early, too late or out of order. NONE: leads to no hazard. Every failure leads to a hazard of Section 4.0, or to none.

## 7.0 Common Causes

A common cause defeats several controls at once and can negate a redundancy [4, Sec. 1.3, p. 3], including the same defect in software that runs in more than one place [1, App. A.1.1, p. 62].

**Wrong onboard clock:** expiry, arm time-outs, scheduled commands, time-tags and the trial time limit depend on it [1, p. 65]. H-4, H-6, H-9, H-11, H-13.
**Shared image code:** start-up code, or a defect common to the current, previous and last-resort firmware, stops all of them. H-1, H-8.
**Data kept over restarts:** a wrong stored setting, counter or flag can stop every image or keep a wrong state [1, p. 64]. H-1, H-6, H-13.
**One power supply:** it feeds both the onboard computer and the controller [5, Sec. 3.3]; one loss stops the controller and the computer that would restore it. H-1, H-8, H-10, H-11, H-15.
**Same data-link software:** one defect at both ends cuts both connections (R-1) [1, p. 66]. H-4.
**One proof of origin:** if one means proves both commands and firmware, its disclosure opens both. H-2, H-5.
**Radiation:** bit errors strike stored firmware, running memory, stored reports, held commands and the arm state (A-5) [1, p. 63]. H-1, H-6, H-7, H-8, H-9, H-10.
**One set of build tools:** a tool defect enters every image. H-1, H-7, H-12.
**Fixed onboard computer software:** it is not updated during the mission [5, Sec. 5.0], so its defects stay. H-1, H-3, H-4.

## 8.0 Hazard Reports

Reports cover the catastrophic and critical hazards, the two categories of the hazard report form [7, Fig. F-1, p. 351]; the others stay in the preliminary hazard list. Each report gives the conditions, worst effect, causes, proposed controls, how the controls fail, and their common causes [2, Sec. 4].

**H-1 The controller is left with no firmware it can run, and cannot be restored.** CATASTROPHIC. Software: CAUSE, CONTROL.
**When:** An update, any restart, or a needed restore. **Effect:** The controller is lost for the rest of the mission (A-1).
**Causes:** (a) power lost while firmware is written (Scenario-600) [1, p. 63]; (b) a bit error in stored firmware (Scenario-610) [1, p. 63]; (c) an update writes over the previous or last-resort firmware [1, p. 65]; (d) new and previous firmware both fail (Scenario-460); (e) the last-resort firmware has changed unnoticed [1, p. 63]; (f) a restart loop never reaches the fall-back [1, p. 64];

(g) the start-up code fails or is damaged [1, p. 63]; (h) the reprogramming link fails, or the onboard computer misses the need to restore (Scenario-520); (i) the onboard computer's known-good copy is damaged [1, p. 63].
**Controls:** (1) an update never writes over the previous or last-resort firmware (Scenario-600) TEST; (2) every image is checked before it runs (Scenario-610) TEST; (3) after a set number of failed starts, the controller returns to its previous firmware (Scenario-440) TEST; (4) a last-resort firmware that updates never replace (Scenario-460) TEST; (5) a restore through the reprogramming link, independent of the controller's software (Scenario-480) TEST; (6) stored firmware and the reprogramming link are checked between uses TEST.
**Fails if:** The image check passes damaged firmware; the failed-start count is lost at a restart; the last-resort firmware, or the image the restore puts back, shares the defect. **Common causes:** Shared image code; data kept over restarts; one power supply; radiation; one set of build tools; fixed onboard computer software.

**H-2 Firmware from someone other than the operator runs on the controller.** CATASTROPHIC. Software: CONTROL.
**When:** An update or a restore. **Effect:** Hostile firmware runs the controller and can refuse updates and report a false state. The restore starts only when no firmware can run (Scenario-480), so this firmware is never removed.
**Causes:** (a) firmware not from the operator arrives (Scenario-410) [1, p. 67]; (b) the means of proving origin is disclosed; (c) a restore starts a stored copy someone else replaced.
**Controls:** (1) the onboard computer refuses firmware not released by the operator (Scenario-410) TEST; (2) the controller checks the origin again before it runs the firmware TEST; (3) the operator can order a restore while firmware runs DEMONSTRATION.
**Fails if:** The check accepts what it should refuse; the means of proof is disclosed. **Common causes:** One proof of origin.

**H-3 New firmware can no longer reach the controller.** CATASTROPHIC. Software: CAUSE, CONTROL.
**When:** After an update is confirmed, or after a fault in the onboard computer. **Effect:** The controller can no longer be corrected for the rest of the mission.
**Causes:** (a) confirmed firmware runs, but cannot receive firmware; (b) a defect in the onboard computer's update path; (c) a wrong stored minimum version or controller identity refuses all firmware [1, p. 67]; (d) firmware storage on the onboard computer stays full [1, p. 66].
**Controls:** (1) new firmware runs a trial before it is confirmed (Scenario-430 to Scenario-450) TEST; (2) it is judged good only after it has received firmware itself TEST; (3) the last-resort firmware can always receive firmware TEST; (4) the operator can order a return to the last-resort firmware DEMONSTRATION.
**Fails if:** The trial skips the update path; the defect is in the onboard computer, out of every control's reach. **Common causes:** Fixed onboard computer software; data kept over restarts.

**H-4 The controller can no longer be commanded or observed.** CATASTROPHIC. Software: CAUSE, CONTROL.
**When:** At any time. **Effect:** The controller runs on its current firmware, but nobody can command or observe it for the rest of the mission.
**Causes:** (a) both data-link connections fail (Scenario-510); (b) one software defect cuts both (R-1) [1, p. 66]; (c) a controller that never stops sending blocks the data link [1, p. 63]; (d) a wrong onboard clock makes every command expire, including the clock correction [1, p. 65]; (e) the ground loses the means of proving origin; (f) a false fault response marks both connections failed (H-13).
**Controls:** (1) two separate data-link connections, one enough (O-3, Scenario-500) TEST; (2) a lost connection is reported (Scenario-510) TEST; (3) the reprogramming link stays usable (Scenario-510) TEST; (4) the clock correction does not depend on the onboard clock TEST.

**Fails if:** One cause cuts both connections. No control yet covers a lost means of proving origin. **Common causes:** Same data-link software; wrong onboard clock; fixed onboard computer software.

**H-5 Someone other than the operator commands the System.** CATASTROPHIC. Software: CONTROL.
**When:** During a contact, or at any time for a source on the spacecraft. **Effect:** Another party can do anything the operator can, and hide it, for the rest of the mission.
**Causes:** (a) a command not from the operator arrives (Scenario-300) [1, p. 67]; (b) the means of proving origin is disclosed; (c) someone else uses the operator interface, and no second person checks a command (A-4); (d) a command arrives from an unexpected system on the spacecraft [1, p. 62].
**Controls:** (1) every command is proven to come from the operator; any other is refused and logged (O-2, Scenario-300) TEST; (2) only the operator can use the operator interface DEMONSTRATION.
**Fails if:** The means of proof is disclosed. **Common causes:** One proof of origin.

**H-6 A command is acted on in a form or at a time the operator did not intend.** CRITICAL. Software: CAUSE, CONTROL.
**When:** While a command is checked, held or run. **Effect:** The controller does what the operator did not intend; the ground learns of it at the next contact.
**Causes:** A command (a) changed in transfer [1, p. 62]; (b) sent again from an old copy (Scenario-300); (c) run twice after a lost answer (Scenario-350); (d) not allowed in the current mode (Scenario-310) [1, p. 63]; (e) with impossible values (Scenario-320) [1, p. 62]; (f) dangerous, without a valid arm (Scenario-330, Scenario-335) [1, p. 63]; (g) past its expiry (Scenario-340) [1, p. 63]; (h) scheduled, run at the wrong time or again after a restart [1, p. 64]; (i) held, or the arm state, changed by a bit error [1, p. 63]; (j) built wrongly by the ground software or entered wrongly by the operator (A-4) [1, p. 63].
**Controls:** (1) the onboard computer checks that every command is new, allowed in the mode, has possible values and has not expired (Scenario-120) TEST; (2) a dangerous command needs an arm, which expires (Scenario-330, Scenario-335) TEST; (3) a repeated command is answered "already done" (Scenario-350) TEST; (4) the ground software checks every command before it is sent (Scenario-120) TEST; (5) every command and answer is logged (Scenario-120) INSPECTION.
**Fails if:** The checks use a wrong clock; a held command or the arm state changes after its check. **Common causes:** Wrong onboard clock; radiation; data kept over restarts.

**H-7 Wrong firmware runs on the controller: damaged, out of date, or with a hidden defect.** CRITICAL. Software: CAUSE, CONTROL.
**When:** During and after an update, a fall-back or a restore. **Effect:** The controller misbehaves until a further update. If the firmware passes its trial, it becomes the known-good firmware the controller falls back to. If it stops updates, H-3 follows.
**Causes:** (a) a hidden defect the trial does not exercise; (b) firmware changed after its check, in transfer or in storage [1, p. 63]; (c) firmware older than the minimum version sent again (Scenario-420); (d) the wrong build, or wrong configuration values, released [1, p. 67]; (e) firmware built for another controller (Scenario-360); (f) a defect added by the build tools; (g) code released without full testing [1, p. 67].
**Controls:** (1) firmware not judged good within the trial time is given up (Scenario-430 to Scenario-450) TEST; (2) the onboard computer refuses firmware that is damaged (Scenario-400), older than the minimum version (Scenario-420) or built for another controller (Scenario-360) TEST; (3) the controller checks firmware again before it runs it (Scenario-610) TEST; (4) the ground software checks every release before it is sent (Scenario-100) TEST; (5) the operator can order a return to the previous firmware DEMONSTRATION.

**Fails if:** The judgement checks too little, or the trial ends before the defect shows; a check is made before the last transfer. **Common causes:** One set of build tools; radiation.

**H-8 The controller stops working between contacts and is not brought back.** CRITICAL. Software: CAUSE, CONTROL, MONITOR.
**When:** In any controller mode. **Effect:** The controller's work stops until the ground acts at the next contact.
**Causes:** (a) a bit error in running memory (Scenario-620) [1, p. 63]; (b) the firmware hangs, deadlocks or overloads [1, p. 63]; (c) a restart does not resume [1, p. 63]; (d) the restart takes longer than the time to the next failure [1, p. 64]; (e) the controller misbehaves when no command arrives for a long time [1, p. 65]; (f) the onboard computer misses the silence, or its watching program has crashed (Scenario-520, Scenario-530).
**Controls:** (1) the controller restarts on its own and resumes (Scenario-620) TEST; (2) the onboard computer notices the silence, tries to bring the controller back and records it (Scenario-520) TEST; (3) a crashed onboard computer program restarts and resumes (Scenario-530) TEST; (4) new firmware that hangs is given up (Scenario-430) TEST.
**Fails if:** The controller's restart and the onboard computer's watch fail from one fault. **Common causes:** Radiation; data kept over restarts; one power supply; shared image code.

**H-9 The ground is shown a false state of the System.** CRITICAL. Software: CAUSE, CONTROL.
**When:** At every contact. **Effect:** The operator decides on false information; a harmful command (H-6) or a missed fault (H-10) follows.
**Causes:** (a) wrong time-tags [1, p. 64]; (b) old reports shown as current [1, p. 63]; (c) a report changed in storage or in transfer [1, p. 62]; (d) "done" or "already done" answered for a command that did not run; (e) the ground software stores or shows a report wrongly [1, p. 62].
**Controls:** (1) the onboard computer time-tags every report (Scenario-110) TEST; (2) every report is checked for damage on arrival TEST; (3) the ground software keeps every report received [5, Sec. 3.2] INSPECTION.
**Fails if:** The onboard clock is wrong; a report is damaged before its check. **Common causes:** Wrong onboard clock; radiation.

**H-10 A fault between contacts never reaches the ground.** CRITICAL. Software: CAUSE, MONITOR.
**When:** Between contacts. **Effect:** The ground does not know of the fault; hidden faults pile up, such as a damaged last-resort firmware (H-1).
**Causes:** (a) the fault is not detected [1, p. 66]; (b) no event report is made [1, p. 65]; (c) the report is lost to full storage, a crash or a power loss (Scenario-210, Scenario-530); (d) the ground loses it.
**Controls:** (1) event reports are kept when storage is full (Scenario-210) TEST; (2) no report is lost without the ground being told (O-5) TEST; (3) stored reports survive a power loss and a restart TEST.
**Fails if:** Fault detection depends on the part that has failed. **Common causes:** One power supply; radiation.

**H-11 A valid command is not acted on, and the ground is not told.** CRITICAL. Software: CAUSE, CONTROL.
**When:** While a command is checked, held or run. **Effect:** The operator believes an action was taken that was not, for example a cancelled update (Scenario-470).
**Causes:** (a) a full buffer drops the command [1, p. 63]; (b) a wrong onboard clock expires it (Scenario-340); (c) a scheduled command is lost at a restart (Scenario-130); (d) data-link traffic blocks it [1, p. 63]; (e) its answer is lost (Scenario-350).
**Controls:** (1) every command gets two answers, received and then done with the result (Scenario-120) TEST; (2) a refusal carries its reason (Scenario-310 to Scenario-340) TEST; (3) scheduled commands survive a restart TEST.
**Fails if:** The answers are lost with the command. **Common causes:** Wrong onboard clock; one power supply.

**H-12 A test function acts outside a test.** CRITICAL. Software: CAUSE.
**When:** In operation. **Effect:** Power is cut, a data-link connection is broken or the radio link is degraded in operation; each is a cause of H-1 or H-4.
**Causes:** (a) fault-injection functions stay in the operating software [1, p. 64]; (b) a wrong command, or one not from the operator, starts them.
**Controls:** (1) test functions serve verification only and are not part of the operating software (Capability-5) INSPECTION.
**Fails if:** A bench build is released for operation [1, p. 67]. **Common causes:** One set of build tools.

## 9.0 Must-Work and Must-Not-Work Functions

The hazard analysis identifies the functions that must work and those that must not work [1, App. A.1.2, p. 62]. Both lists hold in operation.

**Must work:** return to working firmware when new firmware fails (H-1); restore through the reprogramming link (H-1, H-2); check every image before it runs (H-1, H-2, H-7); refuse commands and firmware not from the operator (H-2, H-5); accept the clock correction (H-4); detect and report faults between contacts (H-8, H-10); keep and send down every event report (H-10); answer every command (H-11).

**Must not work:** write over the previous or last-resort firmware (H-1); run firmware not from the operator (H-2); run a command not from the operator (H-5); run a dangerous command without a valid arm, or any command twice (H-6); confirm firmware not judged good (H-7); inject faults (H-12).

## 10.0 Safety-Critical Software

Software is safety-critical if a hazard analysis finds that it meets at least one of the criteria a to e [1, Sec. 4.2, p. 20]. Until the architecture divides each item into components, the whole item is treated as safety-critical [**TBR-3**].

| Software item | Safety-critical | Criteria | Hazards |
|---|---|---|---|
| Controller firmware | YES | a, b, c, e | H-1 to H-4, H-6 to H-13 |
| Onboard computer software | YES | a, b, c, e | H-1 to H-17 |
| Ground software | YES | a, b | H-2, H-4 to H-7, H-9, H-10, H-15, H-17 |
| Bench test tools | NO | NONE | NONE |
| Firmware build tools | NO | NONE | NONE |

**Legend: criteria** [1, Sec. 4.2, p. 20]. a: causes or contributes to a system hazardous condition or event. b: controls functions identified in a system hazard. c: provides mitigation for a system hazardous condition or event. d: mitigates damage if a hazardous condition or event occurs. e: detects, reports and takes corrective action if the system reaches a potentially hazardous state.

***Note.*** *Bench test tools stay out of operation (H-12). Firmware build tools never run in operation; a defect they put into firmware is a common cause (Section 7.0).*

# A Open Items

The values below are not yet known.

**TBD-1** Project name and document reference.

**TBD-2** Likelihood and risk of each hazard, rated once the architecture exists.

The author accepted the severity rule and the meaning of each severity category (Section 3.3) on 10 October 2026. The proposals below await the author's acceptance.

**TBR-1** A catastrophic hazard must need at least three failures together; a critical hazard at least two.

**TBR-2** The hazard list and the severity of each hazard.

**TBR-3** The safety-critical designation of each software item.